

SDR PHY Stack — Algorithm to Air

A one-page reference for the OSU experiments and the UnionLabs abstraction layer.

Overview

We run **one software-defined-radio PHY** (`sdr_system`, C++) under **many algorithms** — MARL random access, federated learning, and the OSU experiments. The goal is a single **abstraction layer** so every algorithm plugs into the PHY the same way: it declares its output **shape and type**, and the PHY grabs, transports, and returns it identically. Everything above the dashed seam below is *algorithm code*; everything below is the *shared PHY*.

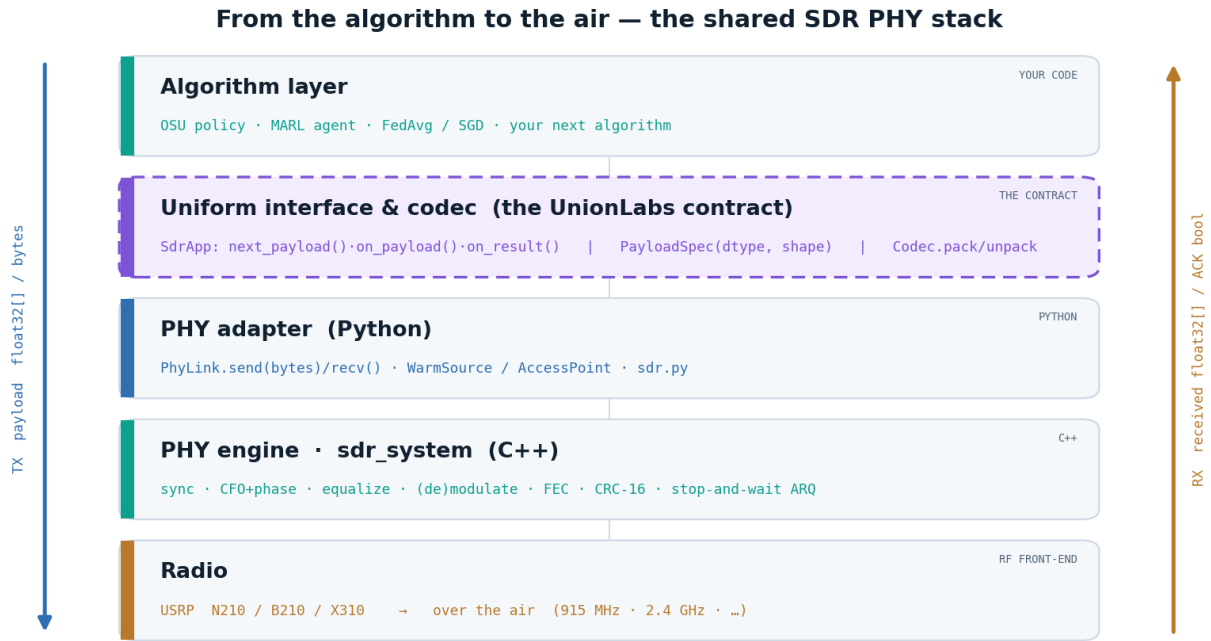


Figure 1: The shared SDR PHY stack: one payload down, one result up.

The stack, layer by layer

Layer	What it is	Boundary type
Algorithm	OSU policy / MARL / FedAvg — produces states, actions, model updates	<code>float32[]</code> (states/data)
Uniform interface & codec	the contract every algorithm implements (the layer to standardize)	<code>PayloadSpec(dtype, shape)</code>
PHY adapter (Python)	<code>PhyLink</code> / <code>WarmSource</code> / <code>AccessPoint</code> / <code>sdr.py</code> — the only Python↔C++ bridge	bytes

Layer	What it is	Boundary type
PHY engine (C++)	sdr_system: sync, CFO/phase, equalize, (de)modulate, FEC, CRC, ARQ	RF waveform
Radio	USRP N210 / B210 / X310	over the air

The contract (the abstraction layer)

Every algorithm implements the **same three methods** — the PHY *pulls* data from it:

- `next_payload()` → `float32[]` — the data to transmit (the PHY grabs this).
- `on_payload(float32[])` — received data handed back (data-transfer apps).
- `on_result(ack: bool)` — delivered / collision (control apps → RL reward).

...and declares **one schema** so shapes never drift on the wire:

- `PayloadSpec(dtype, shape)` — e.g. `float32, (n_params,)`; validated on receive.
- `Codec.pack / unpack` — a single serializer with a self-describing dtype+shape header, so a `float32` vector round-trips exactly and sparse/uint8 payloads stay lossless.

Adding an algorithm = implement three methods + declare one spec. Modulation, coding, ARQ, and the radio come unchanged underneath.

Two archetypes, one interface

	Data-transfer (FL, MNIST)	Control / random-access (MARL, OSU)
Payload	is the data (<code>float32</code> vector)	fixed (content ignored)
Uses	<code>next_payload()</code> + <code>on_payload()</code>	<code>next_payload()</code> + <code>on_result(ack)</code>
Feedback	received data vector	ACK bool = the reward

What the PHY provides every algorithm

Area	Capabilities
Modulation	BPSK, QPSK, 8-PSK, 16/64-QAM, differential (DBPSK/DQPSK/8-DPSK)
Waveform	single-carrier (RRC) and OFDM (pilots track free-running CFO)
FEC	convolutional + Viterbi, LDPC (min-sum), turbo (BCJR); hard/soft; tunable k, iterations
Reliable delivery	stop-and-wait ARQ, CRC-16, ACK over TCP or a second RF path
Channel awareness	energy sensing (listen-before-talk), <code>freq_scan.py</code> , live EVM / pre- & post-FEC BER
Multi-node	N transmitters → 1 access point, per-agent ACK routing, slotted-ALOHA slot clock

Worked example — an OSU policy over the radio

1. Policy observes its **state** [AoI, queue, channel-busy] (float32) and outputs an action: transmit or defer.
2. On *transmit*, the app hands the PHY a payload via `next_payload()`; the adapter frames it and fires **one burst** (`source_arq --max-attempts 1`).
3. The PHY engine modulates (e.g. DQPSK/SC or QPSK/OFDM), codes it (LDPC/turbo), and transmits over the USRP.
4. The access point CRC-checks the burst and routes a per-agent ACK back over TCP.
5. `on_result(ack)` delivers **delivered** / **collision** to the policy → **reward**; state advances. This is exactly the loop MARL uses today — OSU just swaps in its policy.

Deep-dive agenda (code walk, bottom → top)

1. `sdr_system` (C++) — the PHY engine and its `--help` option surface.
2. `sdr.py` — the auto-generated CLI wrapper.
3. `marl_phy.py` / `real_channel.py` — `WarmSource` / `AccessPoint` (today's adapter).
4. `marl_env.py` — observation / action / reward built around the ACK.
5. **proposed** `phy_link.py` — the `SdrApp` + `PayloadSpec` + `Codec` contract.

Then: port OSU onto the contract as the reference implementation; leave the other apps unchanged until it's validated.